Тема 16: Понятие за софтуерно инженерство. Софтуерен процес и модели. Гъвкави методи.

ТЕОРИЯ

1. Софтуерно инженерство — понятие

Софтуер = инструкции + структури от данни + документация.

- Не се произвежда, а се **разработва**
- Не се износва физически, но може да **остарее** (морално)
- Два типа продукти: **общи** (generic, за пазара) и **поръчани** (customized, за конкретен клиент)

Видове софтуер:

Вид	**Описание**	**Примери**
Системен	Обслужва друг софтуер; комуникира с хардуера	ОС, драйвери, компилатори
Приложен	Решава нужди на бизнеса/потребители	Word, Photoshop, медийни плейъри
Научен	Симулации, анализи, автоматизирано проектиране	MATLAB, CAD системи
Вграден (embedded)	Вграден в хардуерни устройства	Firmware в телефони, перални, автомобили
Уеб приложения	Многослойни, многопотребителски	React apps, REST API
ИИ	Нечислови алгоритми за сложни проблеми	Разпознаване на образи, НМ, регресия
Наследен (legacy)	Стар, но критично важен за бизнеса	COBOL банкови системи

Определение: Софтуерното инженерство е дисциплина, занимаваща се с проектирането, разработката, поддръжката и управлението на **висококачествени** софтуерни системи при систематични, дисциплинирани и **измерими** подходи.

Основни аспекти:

- Процес на разработка (жизнен цикъл)
- Технологии и инструменти
- Управление на проекти (планиране, мониторинг)
- Качество (тестване, верификация, валидация)

2. Софтуерен процес

Дефиниция: Последователност от предвидими стъпки и рамка от задачи за изграждане на висококачествен софтуер в срок.

Фази (етапи) на разработка:

Фаза	**Съдържание**
Изисквания	Събиране и анализ на изисквания
Проектиране	Архитектурен и детайлен дизайн
Реализация	Програмиране
Тестване	Unit, интеграционно, системно, потребителско
Внедряване	Инсталиране, конфигурация, предаване на клиента
Поддръжка	Корекции, актуализации, подобрения

Основни дейности (приложими за всеки проект):

1. **Комуникация** — взаимодействие с клиента; събиране на изисквания
2. **Планиране** — рискове, ресурси, времеви график
3. **Моделиране** — Анализ (спецификация на изискванията) + Проектиране (архитектура, интерфейс)
4. **Конструиране** — код + тестване
5. **Внедряване** — доставка + оценка от клиента

Видове модели на софтуерен процес:

Модел	**Описание**
Workflow model	Последователност от дейности с вход/изход
Dataflow model	Трансформации на данни между дейности
Role/Action model	Роли в екипа и техните отговорности

Моделите могат да са:

- **Описателни** — описват как се разработва
- **Предписателни** — предписват как трябва да се разработи

Езици за моделиране (UML и др.):

Използват се за представяне на дейности, роли, артефакти, средства.

По организация: entity-relation, role-interaction, **object-oriented** (UML)

По формалност: формални / полуформални / **неформални**

UML диаграми: Use Case, Class, Sequence, Activity, State, Component, Deployment

Шаблони за описание на процес:

Шаблон = общо решение на общ проблем. Шаблонът на процес трябва да съдържа:

- Начален контекст
- Проблем
- Решение
- Краен контекст
- Свързани шаблони
- Начини на употреба

---

3. Сравнителен анализ на модели на софтуерен процес

3.1 Модел на водопада (Waterfall)

```
Изисквания → Проектиране → Реализация → Тестване → Внедряване → Поддръжка
```

- Линеен последователен подход; всяка фаза завършва преди следващата
- Тежка документация
- **Плюсове**: прост, структуриран, ясни фази
- **Минуси**: трудно за промяна на изисквания; клиентът не вижда резултат до края; проблемите се намират късно
- **Подходящ за**: добре дефинирани изисквания; стабилни технологии; не-критично vrijeme и бюджет

3.2 Прототипен модел (Prototype)

Два вида:

- **Еволюционен прототип** — базов вариант, доразвиван до краен продукт
- **Изхвърлен прототип (throw-away)** — прототипът се представя на клиента за валидиране на изисквания, след което се изхвърля
- **Плюсове**: ранна обратна връзка от клиента; изясняване на изискванията
- **Минуси**: ресурси за прототипа; риск прототипът да стане краен продукт с лош дизайн
- **Подходящ за**: неясни изисквания; неуточнен интерфейс

3.3 Модел на бързата разработка (RAD — Rapid Application Development)

```
Планиране → [Екип 1: Модул A | Екип 2: Модул B | Екип 3: Модул C] → Интеграция
             ←———————————— паралелно ————————————→
```

- Висококоскоростна адаптация на водопада; цел: 60–90 дни
- Паралелна разработка на модули от различни екипи
- Разчита на **готови компоненти** и **автоматично генериране на код**
- **Плюсове**: бързина
- **Минуси**: изисква добро разделяне на модули; за малки проекти
- **Подходящ за**: ясно разграничими модули; малки проекти

3.4 Спираловиден модел (Spiral)

```
        ┌── Планиране
Итерация ├── Анализ на риска
        ├── Разработка + Валидиране
        └── Оценка → следваща итерация
```

- Съчетава водопада и прототипния модел; **акцент на анализ на риска**
- Всяка спирала = планиране + риск + разработка + оценка
- **Плюсове**: управление на риска; гъвкав при промени в изисквания
- **Минуси**: скъп; сложен; изисква опит за оценка на рискове
- **Подходящ за**: големи, сложни проекти с висок риск

3.5 Постъпков (Инкременти) модел (Incremental)

```
Итерация 1: [Core функционалност] → работещ продукт
Итерация 2: [+ функционалност]    → подобрен продукт
Итерация 3: [+ функционалност]    → краен продукт
```

- Системата се разработва и доставя на части (инкременти)
- Всеки инкремент е напълно функционален
- **Плюсове**: ранна доставка на работеща функционалност; риск се разпределя
- **Минуси**: трудно разпределяне на функциите по инкременти
- **Подходящ за**: проекти с ранна нужда от частична функционалност

Сравнителна таблица на моделите

Модел	**Гъвкавост**	**Риск**	**Размер**	**Изисквания**	**Клиент**
Водопад	Ниска	Висок	Малки–средни	Ясни	Малко участие
RAD	Средна	Среден	Малки	Ясни	Умерено
Прототипен	Висока	Нисък	Малки–средни	Неясни	Активно
Спираловиден	Висока	Нисък	Големи	Може да се менят	Умерено

Модел	Гъвкавост	Риск	Размер	Изисквания	Клиент
Инкременти	Висока	Нисък	Всякакви	Може да се менят	Активно

---

4. Гъвкави методи (Agile)

Манифест за гъвкава разработка (Agile Manifesto, 2001) — 4 ценности:

1. **Индивиди и взаимодействие** над процеси и инструменти
2. **Работещ софтуер** над изчерпателна документация
3. **Сътрудничество с клиента** над договорни преговори
4. **Реакция на промяна** над следване на план

12 принципа: ранна и непрекъсната доставка, добре дошли промени, кратки итерации, бизнес и разработчици работят заедно, мотивирани хора, лице в лице комуникация, работещ софтуер е мярка за напредък, устойчиво темпо, техническо съвършенство, простота, самоорганизиращи се екипи, редовна рефлексия.

4.1 Scrum

Scrum = рамка за управление на сложни проекти чрез итеративна и инкрементна разработка.

Роли:

Роля	Отговорности
Product Owner	Определя приоритетите; управлява Product Backlog; гласът на клиента
Scrum Master	Фасилитира процеса; премахва пречки; не е мениджър
Development Team	Самоорганизиращ се екип (3–9 души); разработва продукта

Артефакти:

Артефакт	Описание
Product Backlog	Списък от всички желани функционалности (User Stories), наредени по приоритет
Sprint Backlog	Задачите за текущия Sprint
Increment	Работещ продукт в края на Sprint

Събития (Ceremonies):

Събитие	Честота	Цел
Sprint Planning	Началото на Sprint	Избиране на задачи от Product Backlog
Daily Standup	Всеки ден, ~15 мин	Какво направих? Какво ще правя? Пречки?
Sprint Review	Края на Sprint	Демо на Increment пред Product Owner
Sprint Retrospective	След Review	Как да подобрим процеса?

Sprint: итерация с фиксирана дължина (1–4 седмици); резултат = потенциално доставим Increment.

Scrum процес:

```
Product Backlog → Sprint Planning → Sprint Backlog → Sprint (1-4 седм.) → Increment
                  ←──────────────── повторение ──────────────────→
```

4.2 Extreme Programming (XP)

XP е гъвкав метод с акцент на **технически практики** и качество на кода.

Основни практики:

Практика	Описание
Pair Programming	Двама разработчици работят на едно работно място
Test-Driven Development (TDD)	Пиши теста → провали → имплементирай → мине
Continuous Integration (CI)	Код се интегрира и тества многократно дневно
Refactoring	Непрекъснато подобряване на кода без промяна на функционалност
Small Releases	Чести малки версии
On-site Customer	Клиентът е физически до екипа
Collective Code Ownership	Всеки може да промени всеки код
Simple Design	Прави най-простото, което работи
Coding Standards	Всички следват еднакъв стандарт
Planning Game	Клиент + разработчици планират итерации заедно

Ценности на XP: Комуникация, Простота, Обратна връзка, Смелост, Уважение.

Сравнение Scrum vs XP

Критерий	**Scrum**	**XP**
Фокус	Управление на процеса	Технически практики
Итерации	Sprint (1–4 седм.)	Iteration (1–2 седм.)
Промени	Не по средата на Sprint	Допускат се
Практики	Гъвкаво по отношение на техн.	TDD, Pair Programming, CI задължителни
Роли	PO, SM, Dev Team	Coach, Customer, Developer

Бърза справка

Модел	**Ключова характеристика**
Водопад	Линеен, без връщане назад
RAD	Паралелни екипи, 60–90 дни
Прототипен	Изясняване на изисквания чрез прототип
Спираловиден	Итерации + анализ на риска
Инкременти	Части от продукта се доставят поетапно
Scrum	Sprint, PO/SM/Dev, Backlog, Daily
XP	TDD, Pair Programming, CI, Refactoring
Agile	Хора > процеси; работещ код > документи